

Signal Processing, Machine Learning and Control

HW5

Sravanth Chowdary Potluri
und5uv
CS6762

December 6, 2025

1

- a. **Using P control by itself:** The primary issues with using Proportional (P) control alone are:
 - **Non-zero Steady-State Error:** P control often results in a non-zero steady-state error, particularly when there are disturbance inputs. The controller requires an error to generate a control signal, so it cannot completely eliminate the error in many system types.
 - **Oscillations:** High proportional gains designed to improve response speed or reduce steady-state error can lead to oscillations in the system output.
 - **Design Conflicts:** It is often impossible to fulfill all performance requirements (Stability, Accuracy, Settling Time, Overshoot) simultaneously with a single gain setting K_p . Improving one metric (e.g., accuracy) often degrades another (e.g., overshoot).
- b. **Using I control by itself:** The main problems with using Integral (I) control alone are:
 - **Slow Response:** I control tends to slow down the system's transient response. It reacts to the accumulation of past errors rather than the immediate current error.
 - **Pole Addition:** The integrator adds an open-loop pole at $z = 1$. This often results in a closed-loop pole close to the unit circle boundary, which correlates with the slower response time.
- c. **Using D control by itself:** Derivative (D) control is rarely, if ever, used alone due to the following reasons:
 - **Zero Output for Constant Error:** D control produces an output proportional to the *rate of change* of the error. If the error is constant (even if it is large), the output of a D controller is zero. Consequently, it has a steady-state gain of 0 and cannot maintain the system at a setpoint.

- **Noise Sensitivity:** D control is highly sensitive to noise and stochastic variations. It amplifies high-frequency noise because it differentiates the signal, leading to erratic control outputs.

2

The two methods, Root Locus and Pole Placement Design, are fundamental tools in control system analysis and design, but they differ significantly in their approach, objective, and utility.

1. Root Locus (RL) Method:

- **Objective:** The primary objective is analysis and observation. It is used to plot the trajectories of the closed-loop poles as a single controller parameter (typically the proportional gain, K) is varied from zero to infinity.
- **Approach:** It is a graphical method based on the characteristic equation of the closed-loop system: $1 + KD(z)G(z) = 0$. The roots of this equation are plotted as K varies, revealing the root's locations and whether the system remains stable.
- **Outcome/Utility:** It shows the entire range of possible closed-loop pole locations and, crucially, the range of gain K for which the system is stable. It allows the designer to observe how performance characteristics (like stability, settling time, and overshoot) are affected by changing the gain.

2. Pole Placement (PP) Design:

- **Objective:** The primary objective is direct synthesis. It aims to determine the specific values of all control parameters (e.g., K_p , K_i , K_d) required to place the closed-loop poles at a set of pre-specified, desired locations.
- **Approach:** It is an analytical method that involves coefficient matching. It requires constructing a desired characteristic polynomial based on the target performance specifications (SASO properties) and equating it to the modeled characteristic polynomial of the closed-loop system, which is a function of the controller gains.
- **Outcome/Utility:** It directly yields the controller gain values necessary to meet the desired closed-loop properties, such as a specific settling time and overshoot, by positioning the poles at precise locations within the unit circle.

Summary of Difference: Root Locus is an **observational/diagnostic** tool that shows *what happens* to the poles as the gain changes. Pole Placement is a **prescriptive/synthesis** tool that determines *what gain is needed* to achieve specific pole locations.

3

Given the open-loop transfer function $G(z) = \frac{0.5}{z-0.75}$ and a proportional controller K_p , the closed-loop characteristic equation is determined by $1 + K_p G(z) = 0$.

$$1 + K_p \left(\frac{0.5}{z - 0.75} \right) = 0 \implies z - 0.75 + 0.5K_p = 0 \quad (1)$$

The closed-loop pole is $p = 0.75 - 0.5K_p$.

- a. **Stability:** For the system to be stable, the pole must lie inside the unit circle, i.e., $|p| < 1$.

$$\begin{aligned} |0.75 - 0.5K_p| &< 1 \\ -1 &< 0.75 - 0.5K_p < 1 \\ -1.75 &< -0.5K_p < 0.25 \\ -0.5 &< K_p < 3.5 \quad (\text{Condition 1}) \end{aligned}$$

- b. **Steady-state error** $e_{ss} < 0.1$: For a unit step reference, the steady-state error is given by $e_{ss} = \frac{1}{1+K_p G(1)}$. First, calculate the DC gain of the plant $G(1)$:

$$G(1) = \frac{0.5}{1 - 0.75} = \frac{0.5}{0.25} = 2$$

Now, apply the condition $e_{ss} < 0.1$:

$$\begin{aligned} \frac{1}{1 + K_p(2)} &< 0.1 \\ 1 &< 0.1(1 + 2K_p) \quad (\text{assuming stable } K_p > -0.5) \\ 10 &< 1 + 2K_p \\ 9 &< 2K_p \\ K_p &> 4.5 \end{aligned}$$

This condition contradicts Condition 1 for stability. Therefore, there is **no value of K_p that can satisfy both stability and steady-state error requirements simultaneously**.

- c. **Settling time** $k_s < 10$: The approximate settling time is given by $k_s \approx \frac{-4}{\ln |p|}$.

$$\begin{aligned} \frac{-4}{\ln |p|} &< 10 \\ \ln |p| &< -0.4 \quad (\text{since } \ln |p| \text{ is negative}) \\ |p| &< e^{-0.4} \approx 0.6703 \end{aligned}$$

Solving for K_p :

$$\begin{aligned} |0.75 - 0.5K_p| &< 0.6703 \\ -0.6703 &< 0.75 - 0.5K_p < 0.6703 \\ -1.4203 &< -0.5K_p < -0.0797 \\ 0.1594 &< K_p < 2.8406 \end{aligned}$$

- d. **Maximum Overshoot** $M_p < 0.1$: For a first-order system, overshoot depends on the location of the closed-loop pole p :

- If $0 \leq p < 1$: The response is non-oscillatory, so $M_p = 0$.
- If $-1 < p < 0$: The response oscillates, and $M_p = |p|$.
- If $|p| \geq 1$: The system is unstable or marginally stable.

Step 1: Analyze Positive Poles ($p \geq 0$)

For the pole to be positive:

$$0.75 - 0.5K_p \geq 0 \implies 0.75 \geq 0.5K_p \implies K_p \leq 1.5$$

In this region, $M_p = 0$, which is strictly less than 0.1. However, we must also ensure the system is stable ($p < 1$). From part (a), the stability limit is $K_p > -0.5$.

$$\therefore \text{Valid range for positive poles: } -0.5 < K_p \leq 1.5$$

Step 2: Analyze Negative Poles ($p < 0$)

For the pole to be negative:

$$0.75 - 0.5K_p < 0 \implies K_p > 1.5$$

Here, the overshoot is $M_p = |p|$. We require $|p| < 0.1$:

$$|0.75 - 0.5K_p| < 0.1$$

Since the pole is negative, this is equivalent to:

$$\begin{aligned} -0.1 &< 0.75 - 0.5K_p < 0 \\ -0.85 &< -0.5K_p < -0.75 \\ 1.5 &< K_p < 1.7 \end{aligned}$$

$$\therefore \text{Valid range for negative poles: } 1.5 < K_p < 1.7$$

Step 3: Combine Ranges

Combining the valid ranges from Step 1 and Step 2:

$$(-0.5 < K_p \leq 1.5) \cup (1.5 < K_p < 1.7)$$

$$-0.5 < K_p < 1.7$$

4

The two methods for determining the coefficients of a PI controller (K_p and K_i) are fundamentally different in their approach, reliance on system knowledge, and the trade-offs involved.

1. Experimental Tuning (Trial-and-Error or Heuristic Methods):

- **Basis:** The coefficients are determined by applying the PI controller to the physical system and observing the closed-loop response (e.g., to a step input). Tuning methods like Ziegler-Nichols (though not detailed here) or simple trial-and-error are used.

- **System Knowledge:** This method implicitly handles the system's real-world non-linearities, unmodeled dynamics, and practical constraints (e.g., actuator limits) since it operates directly on the plant. It does not require a precise mathematical transfer function $G(z)$.
- **Objective:** To adjust K_p and K_i until the measured performance metrics (e.g., settling time, overshoot, steady-state error) are satisfactory.
- **Limitations:** It is often time-consuming, requires repeated tests on the actual hardware, and carries the risk of destabilizing the system during the tuning process if inappropriate gains are chosen. The resulting gains may be acceptable but rarely optimal.

2. Pole Placement Design:

- **Basis:** This is an **analytical synthesis method** that requires a precise, linear mathematical model (transfer function $G(z)$) of the system.
- **Objective:** To algebraically calculate the exact controller coefficients (K_p, K_i) that force the closed-loop system poles to pre-selected, desired locations within the unit circle. These locations are chosen to guarantee specific performance characteristics.
- **Procedure:** The designer sets the desired characteristic polynomial, computes the modeled characteristic polynomial (in terms of K_p and K_i), and then solves for the gains by coefficient matching.
- **Limitations:** The performance is entirely dependent on the **accuracy of the model**. If the real system deviates significantly from the model, the analytically calculated gains may not yield the desired performance in the physical plant.

Summary: Pole Placement is a direct, model-based **synthesis** technique guaranteeing specific pole locations. Experimental tuning is an indirect, hardware-based **analysis and adjustment** technique that accounts for real-world effects but relies on repeated observation.

5

1. **Integral (I) Controller:** The transfer function for the Integral controller is:

$$C_I(z) = \frac{U(z)}{E(z)} = \frac{K_I z}{z - 1}$$

2. **Proportional-Derivative (PD) Controller:** The PD controller consists of a proportional term and a derivative term. Based on the discrete implementation where the derivative is the rate of change of the error (backward difference), the transfer function is:

$$C_{PD}(z) = \frac{U(z)}{E(z)} = K_p + K_d \frac{z - 1}{z}$$

This can also be combined into a single fraction:

$$C_{PD}(z) = \frac{K_p z + K_d(z - 1)}{z}$$

6

The design of a Proportional-Integral (PI) controller using the pole placement method involves a systematic process to determine the controller gains (K_p and K_I) that satisfy specific performance requirements. The main steps are:

1. **Compute the desired closed-loop poles:** Determine the locations of the desired closed-loop poles based on the system's performance specifications, such as settling time (k_s) and maximum overshoot (M_p).
2. **Construct and expand the desired characteristic polynomial:** Create the characteristic polynomial from the desired poles determined in Step 1. For example, if the desired poles are p_1 and p_2 , the polynomial is $(z - p_1)(z - p_2) = 0$. Expand this to find the desired coefficients.
3. **Construct and expand the modeled characteristic polynomial:** Derive the actual closed-loop characteristic equation of the system, which includes the unknown controller gains K_p and K_I . This is done by analyzing the closed-loop transfer function containing the plant $G(z)$ and the PI controller.
4. **Solve for K_p and K_I (Coefficient Matching):** Equate the coefficients of the modeled characteristic polynomial (from Step 3) to the coefficients of the desired characteristic polynomial (from Step 2). Solve the resulting system of linear equations to find the required values for K_p and K_I .
5. **Verify the results:** Check the validity of the design. This involves:
 - Ensuring the closed-loop poles lie within the unit circle for stability.
 - Simulating the transient response to verify that the design goals (e.g., zero steady-state error, specific settling time) are met.

7

To implement a PID controller on a microprocessor, the algorithm typically follows a cyclic process of reading inputs, calculating the three control terms (Proportional, Integral, Derivative), summing them, and updating the state variables for the next iteration.

Main Instructions

1. **Initialize:** Define variables for gains (K_p, K_i, K_d), setpoint, error sums, and previous error state.
2. **Read Sensors:** Obtain the current measured value (Input) from the system sensors.
3. **Compute Time Step:** Calculate the time difference (dt) since the last cycle to ensure accurate integration and differentiation.
4. **Calculate Error:** Compute the difference between the desired Setpoint and the current Input.

5. **Calculate Integral Term:** Accumulate the error over time (Integration) by adding the current error multiplied by dt to the running sum.
6. **Calculate Derivative Term:** Determine the rate of change of the error by subtracting the previous error from the current error and dividing by dt .
7. **Compute Output:** Sum the P, I, and D terms:

$$\text{Output} = (K_p \cdot \text{Error}) + (K_i \cdot \text{Integral}) + (K_d \cdot \text{Derivative})$$
8. **Update State:** Save the current error and current time for use in the next loop iteration.
9. **Actuate:** Send the computed Output signal to the actuator.
10. **Loop:** Repeat the process at the defined sampling rate.

Pseudocode

```
// Define Variables
unsigned long lastTime;
double Input, Output, Setpoint;
double errSum, lastErr;
double kp, ki, kd;

void Compute() {
    // 1. Determine time elapsed
    unsigned long now = millis();
    double timeChange = (double)(now - lastTime);

    // 2. Compute errors
    double error = Setpoint - Input;
    errSum += (error * timeChange);           // Integral term
    double dErr = (error - lastErr) / timeChange; // Derivative term

    // 3. Compute PID Output
    Output = kp * error + ki * errSum + kd * dErr;

    // 4. Update variables for next iteration
    lastErr = error;
    lastTime = now;
}
```

8

1. Define the System Components:

- **Plant Transfer Function:**

$$G(z) = \frac{0.47}{z - 0.43}$$

- PI Controller Transfer Function:** The transfer function for the PI controller is the sum of the proportional term (K_p) and the integral term. Based on the slides, the integral term is $\frac{K_i z}{z-1}$.

$$K(z) = K_p + \frac{K_i z}{z-1}$$

Substituting the given values $K_p = 2$ and $K_i = 1$:

$$K(z) = 2 + \frac{1 \cdot z}{z-1} = \frac{2(z-1) + z}{z-1} = \frac{2z-2+z}{z-1} = \frac{3z-2}{z-1}$$

2. Calculate the Open-Loop Transfer Function ($L(z)$):

$$L(z) = K(z)G(z) = \left(\frac{3z-2}{z-1} \right) \left(\frac{0.47}{z-0.43} \right)$$

Multiplying the numerators and denominators:

$$\text{Numerator} = 0.47(3z-2) = 1.41z - 0.94$$

$$\text{Denominator} = (z-1)(z-0.43) = z^2 - 0.43z - 1z + 0.43 = z^2 - 1.43z + 0.43$$

$$L(z) = \frac{1.41z - 0.94}{z^2 - 1.43z + 0.43}$$

3. Calculate the Closed-Loop Transfer Function ($F_r(z)$): Assuming no transducer ($H(z) = 1$), the closed-loop transfer function is:

$$F_r(z) = \frac{Y(z)}{R(z)} = \frac{L(z)}{1 + L(z)}$$

Let $N(z) = 1.41z - 0.94$ and $D(z) = z^2 - 1.43z + 0.43$.

$$F_r(z) = \frac{\frac{N(z)}{D(z)}}{1 + \frac{N(z)}{D(z)}} = \frac{N(z)}{D(z) + N(z)}$$

Compute the new denominator $D_{CL}(z) = D(z) + N(z)$:

$$\begin{aligned} D_{CL}(z) &= (z^2 - 1.43z + 0.43) + (1.41z - 0.94) \\ &= z^2 + (-1.43 + 1.41)z + (0.43 - 0.94) \\ &= z^2 - 0.02z - 0.51 \end{aligned}$$

The closed-loop transfer function is:

$$F_r(z) = \frac{1.41z - 0.94}{z^2 - 0.02z - 0.51}$$

9

The specific computing system issues that present challenges to or deviations from classical feedback control theory include:

- **Bounded Output and Limit Cycles:** Classical control theory typically defines instability as "BIBO" (Bounded Input, Bounded Output), where an unstable system's output grows towards infinity. However, in computing systems, metrics like CPU utilization are physically bounded between 0 and 1 (0% to 100%). Because the "physics" of the system do not permit unbounded growth, instability in these systems does not manifest as an infinite signal but rather as a **limit cycle**, where the value bounces back and forth (e.g., between zero and one).
- **Stochastic Resource Consumption:** Computing systems exhibit substantial variability due to the stochastic nature of resource consumption (e.g., variable response times, queue lengths, and utilizations). This stochastic behavior often requires the addition of components like **Moving-Average Filters** to mitigate noise, which alters the system dynamics (adding delay and poles) in ways that simple proportional control models do not inherently address without modification.
- **Measurement Delays:** While classical theory can model delays, computing systems (like the IBM Lotus Domino Server) often have specific measurement delays (e.g., 2 time units). These delays fundamentally change the root locus and stability analysis compared to simple, delay-free examples, requiring specific compensation or more complex analysis (like the addition of poles at zero).

10

Reinforcement Learning (RL) is a paradigm of machine learning concerned with how an intelligent agent ought to take actions in an environment in order to maximize the notion of cumulative reward. Unlike supervised learning, where the agent is told the correct action to take, an RL agent must discover which actions yield the most reward by trying them.

The core components of an RL system are:

- **Agent and Environment:** The agent interacts with the environment in discrete time steps. At each step t , the agent receives a state S_t and selects an action A_t .
- **Reward Signal:** The environment responds to the action with a new state S_{t+1} and a numerical reward R_{t+1} . The goal is to maximize the expected sum of discounted future rewards.
- **Policy (π):** The strategy used by the agent to determine the next action based on the current state. It is a mapping from states to actions.
- **Exploration vs. Exploitation:** A fundamental dilemma where the agent must balance exploring new actions to find higher rewards (exploration) versus selecting the best-known actions to gain reward (exploitation).

Comparison with Deep Neural Networks (Deep Learning)

While often used together, Reinforcement Learning and Deep Neural Networks (DNNs) address different problems:

- **Learning Objective:** Standard Deep Learning (Supervised Learning) focuses on function approximation and pattern recognition. It minimizes the error between a predicted output and a known "ground truth" label using a static dataset. In contrast, RL focuses on sequential decision-making. It maximizes a scalar reward signal over time in a dynamic environment where the data distribution changes as the agent learns.
- **Feedback Mechanism:** In DNNs, feedback is instantaneous and corrective (via backpropagation of loss). In RL, feedback (reward) is often delayed and sparse; an action taken now might not yield a positive reward until many steps later (the credit assignment problem).
- **Integration (Deep RL):** Deep Neural Networks are often employed *within* RL systems (Deep RL) to approximate the policy or value functions when the state space is too large to store in a table (e.g., using a Convolutional Neural Network to process video frames as the state).

Comparison with Feedback Control

Reinforcement Learning and Classical Feedback Control (e.g., PID) both deal with controlling systems, but their approaches differ significantly:

- **Model Dependency:** Classical Feedback Control typically relies on a mathematical model of the system (the plant, $G(z)$) to design the controller. Techniques like Root Locus and Pole Placement require knowledge of poles and zeros to guarantee stability and performance properties (SASO). RL is fundamentally "model-free"; the agent learns the physics of the environment purely through trial-and-error interaction, without requiring an analytical transfer function.
- **Design vs. Learning:** Feedback controllers use fixed parameters (like K_p, K_i, K_d) derived analytically to meet specific transient and steady-state requirements. RL "learns" its control policy (essentially its internal gains) automatically over time. While Feedback Control prioritizes mathematical guarantees of stability and safety, RL prioritizes optimality of the long-term reward, sometimes at the cost of training stability or safety during the exploration phase.
- **Adaptability:** A standard PID controller is linear and time-invariant. It may fail if the system dynamics change significantly unless adaptive control methods are added. RL agents can theoretically adapt to non-linear and changing environments by continuously updating their policy based on new reward feedback.

References

- [1] *CS 6762: Signal Processing, Machine Learning and Control Class Materials and Lectures*. University of Virginia.
- [2] *Other Internet Sources for basic information, facts and examples*.

Acknowledgment

This Homework makes use of Generative AI models like Gemini and ChatGPT for research to form and refine answers as required for this homework and some of the main sources for the information are cited. The author acknowledges that they have not given or received unauthorized assistance on this homework. The Author also acknowledges that they have understood the course content and lectures for the preparation of these homework solutions.